

TEAM01_PROJECT2_FINAL_REPORT

-RISC-V Processor 와 Vision Sensor 를 활용한 FPGA 기반 Fitness Runner 시스템 보고서-

전자공학과

차상훈

전자공학과

목차

1. Application 아이디어 및 목표
2. 전체 시스템 구조
3. RISC-V processor 활용 방식
4. Vision sensor interface 및 데이터 처리 방식
5. FPGA logic 과 software 의 역할 분담
6. Agent AI 활용 내용
7. 구현 과정에서 발생한 문제와 해결 방법
8. Demo 결과
9. 한계점 및 개선 방향
10. 팀원별 역할 분담

1. Application 아이디어 및 목표

본 프로젝트의 목표는 FPGA 보드, RISC-V soft processor, HuskyLens AI Vision Sensor, Raspberry Pi 기반 웹 인터페이스를 연동하여 사용자의 신체 움직임을 실시간 게임 입력으로 변환하는 임베디드 피트니스 응용 시스템을 구현하는 것이다. 기존의 키보드나 마우스 기반 입력 방식과 달리, 본 시스템은 사용자의 얼굴 위치를 비전 센서로 추적하고, FPGA 내부에서 좌표 데이터를 처리한 뒤, RISC-V processor가 해당 정보를 읽어 게임 입력 프레임으로 변환한다. 이후 Raspberry Pi는 UART로 수신한 데이터를 웹 기반 피트니스 게임 화면에 반영한다.

초기 아이디어는 HuskyLens가 제공하는 object 또는 face block data 중 xCenter 값을 이용하여 사용자가 화면의 왼쪽, 중앙, 오른쪽 중 어디에 위치하는지 판단하는 중앙 정렬 가이드 시스템이었다. 이후 프로젝트를 확장하여 xCenter 뿐만 아니라 yCenter의 변화량까지 활용하고, 이를 통해 좌우 이동과 점프 동작을 게임 입력으로 변환하는 시스템으로 발전시켰다.

최종 Application은 "FPGA Fitness Runner"로 정의하였다. 사용자는 HuskyLens 앞에서 몸을 좌우로 이동하거나 위아래로 움직이고, 시스템은 이를 실시간으로 게임 캐릭터의 좌우 이동 및 점프 동작으로 변환한다. 웹 화면에서는 목표 칼로리, 세트 수, 세트당 목표 칼로리, 휴식 시간 등을 설정할 수 있으며, 운동 동작에 따라 캐릭터가 장애물을 피하고 칼로리 점수가 증가한다.

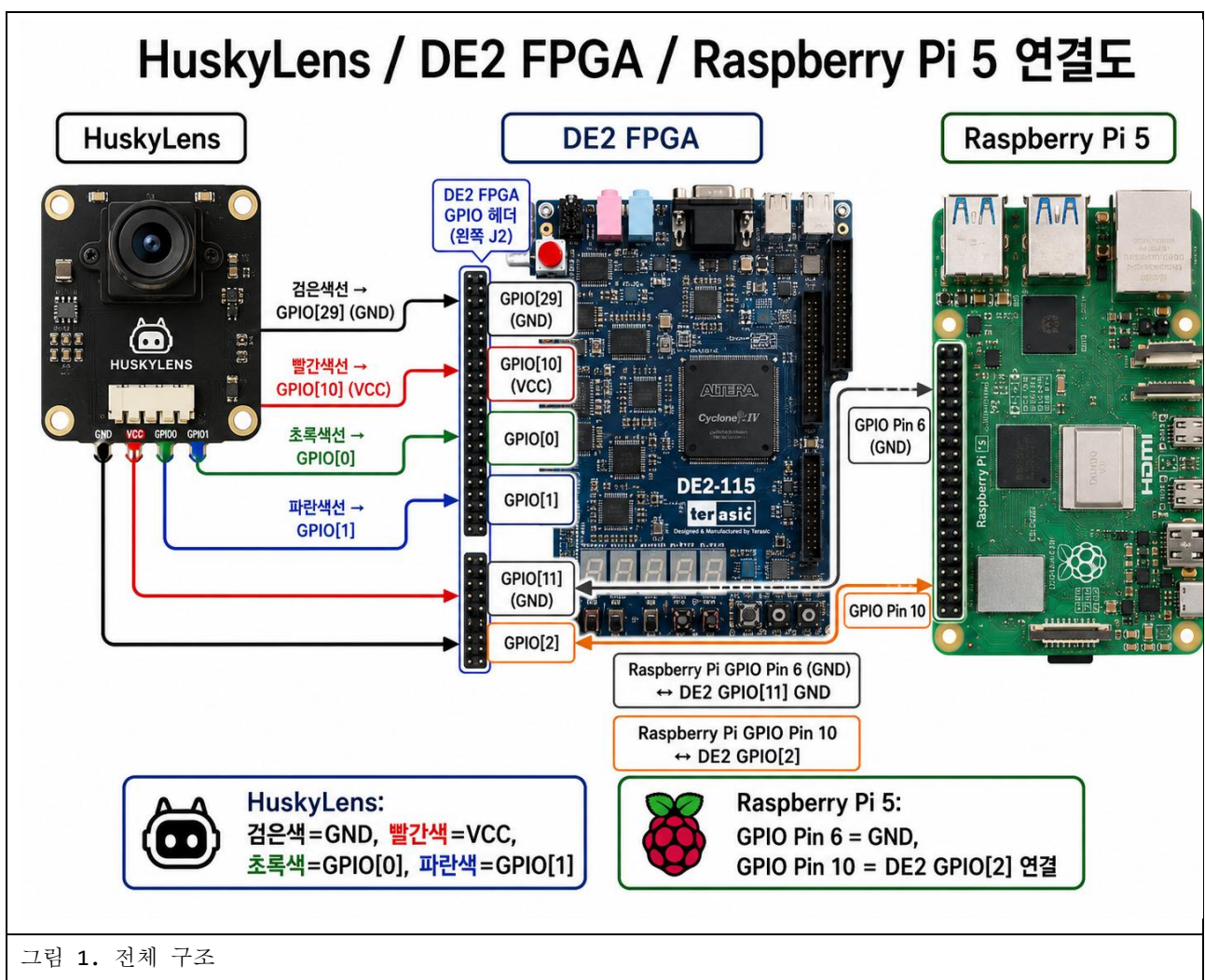
1.1 세부 목표

1. HuskyLens와 FPGA 간 UART 통신을 구현하여 vision sensor에서 제공하는 사람 위치 데이터를 수신한다.
2. FPGA 내부에 RISC-V 32-bit processor를 통합하고, processor가 MMIO 방식으로 HuskyLens의 상태 및 좌표 데이터를 읽을 수 있도록 한다.
3. RISC-V software가 xCenter, yCenter, y 변화량을 이용해 게임 입력 프레임을 생성하고, 이를 UART로 Raspberry Pi 또는 PC에 전송한다.
4. Raspberry Pi에서 Python과 Flask 기반 웹 서버를 실행하여 FPGA에서 수신한 데이터를 실시간 게임 UI로 시각화한다.
5. 단순한 하드웨어 통신 실험을 넘어, FPGA logic, RISC-V software, vision sensor, 웹 UI가 하나의 응용 시스템으로 동작하도록 통합한다.

2. 전체 시스템 구조

전체 시스템은 크게 네 부분으로 구성된다. 첫 번째는 사용자의 움직임을 인식하는 HuskyLens AI Vision Sensor, 두 번째는 센서와 processor를 연결하는 FPGA logic, 세 번째는 FPGA 내부에서 실행되는 RISC-V processor 및 memory system, 네 번째는 Raspberry Pi 또는 PC에서 동작하는 Python/Flask 기반 게임 UI이다.

사용자 움직임
 ↓
 HuskyLens AI Vision Sensor
 ↓ UART 9600 baud, 8N1
 FPGA HuskyLens UART Receiver / Parser
 ↓
 MMIO Register
 ↓
 RISC-V 32-bit Processor
 ↓
 UART TX Frame Generation
 ↓
 Raspberry Pi / PC Python Receiver
 ↓
 Flask Web Game UI



HuskyLens 는 사용자의 얼굴 또는 object block 을 감지하고, 해당 block 의 중심 좌표인 xCenter 와 yCenter 를 UART 로 FPGA 에 전달한다. FPGA 내부의 huskylens_core 는 HuskyLens 와의 UART 송수신을 담당하며, knock command, request command, response frame 수신, parser 처리, timeout 판단 등을 수행한다.

FPGA top module 에서는 RISC-V processor, instruction memory, data memory, MMIO, HuskyLens core 가 하나의 시스템으로 연결된다. RISC-V processor 는 RISCV 를 wrapper 로 사용하고, 내부 datapath 는 DATAPATH, control path 는 CONTROLLER, MAINDEC, ALUDEC 로 구성된다. 또한 register file, immediate extension, ALU, branch prediction module 이 processor 를 구성한다.

2.1 주요 모듈 구성

분류	파일/모듈	역할
Processor	RISCV.v	RISC-V processor top wrapper
Datapath	DATAPATH.v	5-stage pipeline, forwarding, load-use stall 처리
Controller	CONTROLLER.v / MAINDEC.v / ALUDEC.v	명령어 decode 및 제어 신호 생성
Execution	alu.v / REGFILE.v / EXTEND.v	ALU 연산, register file, immediate 생성
Branch	BRANCH_PRED.v	2-bit BHT dynamic branch predictor
Memory	IMEM.v / DMEM.v	instruction memory 및 data memory
I/O	MMIO.v	HuskyLens 상태 read 및 UART TX peripheral 제공
Vision Interface	huskylens_core.v	HuskyLens UART 송수신, frame parsing
System Top	TOP.v	전체 module integration 및 외부 pin 연결
Raspberry Pi UI	rpi_uart_receive.py	UART frame 수신 및 Flask web game UI 표시

2.2 메모리 맵

주소	방향	내용
0x0000_0000~0x0000_3FFC	R	IMEM, PC fetch 영역
0x0000_0000~0x0000_3FFC	R/W	DMEM, load/store 영역
0x4000_0000	R	HuskyLens status [10]Done [9]No_Person [8]Fail [7]Block [6]OK
0x4000_0004	R	X center [15:0]
0x4000_0008	R	Y center [15:0]
0x4000_000C	R	마지막 수신 byte [7:0]
0x4000_0020	R	UART TX busy [0]
0x4000_0024	W	UART TX data [7:0]

2.3 외부 연결 구조

신호	핀/연결	용도
CLOCK_50	PIN_N2	50 MHz system clock

KEY[0]	PIN_G26	reset, active-low
KEY[1]	PIN_N23	HuskyLens 통신 시작
GPIO_0[0]	PIN_D25	HuskyLens TX → FPGA RX
GPIO_0[1]	PIN_J22	FPGA TX → HuskyLens RX
GPIO_0[2]	사용 가능 GPIO	FPGA TX → Raspberry Pi GPIO15(RXD) 게임 데이터 송신
UART_TXD	PIN_B25	PC RS-232 또는 외부 UART debug 경로

3. RISC-V processor 활용 방식

본 프로젝트의 핵심은 FPGA 내부에 RISC-V 32-bit soft processor 를 단순히 탑재하는 데 그치지 않고, sensor data processing 과 output frame generation 에 활용했다는 점이다. RISC-V processor 는 5-stage pipeline 구조를 기반으로 하며, forwarding unit 과 load-use hazard stall 처리를 포함한다. 또한 branch misprediction 발생 시 flush 를 수행하여 잘못 fetch 된 instruction 을 제거한다.

Branch prediction 은 32-entry 2-bit saturating counter 기반 dynamic predictor 를 사용한다. PC[6:2]를 index 로 사용하며, 각 entry 는 strong not-taken, weak not-taken, weak taken, strong taken 상태를 갖는다. 반복 루프가 많은 sensor polling software 에서는 branch 가 자주 발생하므로, 동적 branch predictor 를 통해 반복 루프의 예측 효율을 높일 수 있다.

3.1 MMIO 기반 sensor data 접근

RISC-V processor 는 일반 memory 접근과 MMIO 접근을 구분하여 동작한다. 주소가 0x4000_0000 영역에 해당하면 MMIO 로 판단하고, HuskyLens 상태 또는 UART TX peripheral 에 접근한다.

Processor 가 HuskyLens 데이터를 읽는 과정은 다음과 같다.

1. 0x4000_0000 에서 status register 를 읽는다.
2. Block_Seen 또는 Done 상태를 확인한다.
3. 0x4000_0004 에서 xCenter 를 읽는다.
4. 0x4000_0008 에서 yCenter 를 읽는다.
5. 이전 yCenter 와 현재 yCenter 의 차이를 이용해 delta 를 계산한다.
6. xCenter 를 게임 화면에 맞는 0~159 범위로 scaling 한다.
7. delta 와 위치 조건을 기반으로 action 을 결정한다.
8. UART TX busy 가 0 이 될 때까지 0x4000_0020 을 polling 한다.
9. 0x4000_0024 에 1 바이트씩 write 하여 frame 을 전송한다.

3.2 RISC-V software 출력 frame

RISC-V software 가 생성하는 최종 게임 입력 프레임은 4 바이트로 구성된다. Raspberry Pi 의 Python 수신 프로그램은 0xFF header 를 기준으로 payload 3 바이트를 읽고, x 위치, y 변화량, action 값을 web game state 로 반영한다.

Byte index	값	의미
0	0xFF	frame header
1	X_scaled	0~159 범위로 정규화된 좌우 위치
2	delta + 128	signed y 변화량을 unsigned byte 로 전송
3	action	0=RUN, 1=JUMP

또한 KEY[2] 입력을 이용한 서버 종료 frame 도 정의하였다. 이 frame 은 0xFE, 'S', 'T', 'P'로 구성되며, Raspberry Pi 수신 프로그램은 이를 server stop request 로 해석한다.

4. Vision sensor interface 및 데이터 처리 방식

Vision sensor 로는 HuskyLens AI Vision Sensor 를 사용하였다. HuskyLens 는 내부적으로 얼굴 인식, object tracking, object recognition 등의 기능을 제공하며, 본 프로젝트에서는 가장 안정적인 운용을 위해 Face Detection 모드를 중심으로 사용하였다. Object Tracking 모드도 고려했지만, 실제 환경에서는 구도와 거리 변화에 민감하여 인식 안정성이 떨어졌고, 얼굴이 화면에 잡히는 경우 Face Detection 이 더 안정적으로 좌표 데이터를 제공하였다.

HuskyLens 와 FPGA 는 TTL UART 방식으로 연결하였다. 통신 설정은 9600 baud, 8N1 이며, FPGA 의 50 MHz clock 기준으로 CLKS_PER_BIT = 5208 을 사용하였다. 중요한 점은 HuskyLens 의 UART 는 TTL level UART 이므로, DE2 보드의 DB9 RS-232 포트에 직접 연결하면 안 된다는 것이다. RS-232 는 전압 레벨과 신호 방식이 다르므로, HuskyLens 는 반드시 FPGA GPIO pin 에 연결해야 한다.

4.1 HuskyLens 통신 FSM

```
IDLE
→ SEND_KNOCK
→ WAIT_OK
→ SEND_REQUEST
→ WAIT_RESPONSE
→ PARSE_RESPONSE
→ DECIDE
→ SAMPLE_GAP
→ FAIL
```

KEY[1]을 누르면 FPGA 는 HuskyLens 에 knock command 를 전송한다. HuskyLens 가 OK response 를 반환하면 FPGA 는 object 또는 face block data 를 요청하는 command 를 보낸다. 이후 response

frame 을 수신하고 parser 가 payload 에서 xCenter, yCenter, width, height, ID 등 block data 를 추출한다.

4.2 좌표 기반 동작 판별

초기 구현에서는 xCenter 만 사용하여 사용자의 좌우 위치를 판단하였다. 이후 게임 응용에서는 xCenter 를 좌우 이동 입력으로 사용하고, yCenter 의 변화량을 동작 판별에 활용하였다. yCenter 는 카메라 화면에서 사용자의 얼굴 중심 위치를 나타내며, 이전 frame 과 현재 frame 의 차이를 이용하면 상하 이동 속도를 추정할 수 있다.

```
X_scaled = xCenter / 2      (0~159 범위로 연속 매핑)
delta    = 현재 yCenter - 이전 yCenter
```

```
JUMP 판정: delta < -15 AND yCenter < baseline x 3/4
           (속도 조건과 위치 조건을 함께 만족할 때만 점프)
```

```
최종 frame: [0xFF][X_scaled][delta+128][action]
```

최종 단계에서는 y 방향 속도와 위치 조건을 함께 사용하여 점프 오탐을 줄였다. 단순히 y 변화량만 보고 점프를 판단하면 사용자가 살짝 움직이거나 카메라가 흔들려도 점프로 인식될 수 있기 때문이다. 따라서 빠르게 위로 이동하는 속도 조건과 기준 위치보다 충분히 높은 위치에 도달했는지의 위치 조건을 함께 고려하였다.

5. FPGA logic 과 software 의 역할 분담

본 프로젝트에서는 전체 기능을 모두 hardware 로 구현하지 않고, FPGA logic 과 RISC-V software 의 역할을 분리하였다. 이 구조는 유지보수성과 확장성을 높이는 데 중요한 역할을 한다.

5.1 FPGA logic 역할

- HuskyLens 와의 UART 송수신을 담당한다.
- HuskyLens protocol frame 의 header, command, payload 를 해석한다.
- xCenter, yCenter, status, last RX byte 를 MMIO register 로 제공한다.
- RISC-V 가 byte 단위로 데이터를 전송할 수 있도록 UART TX peripheral 을 제공한다.
- button input, reset, timeout, fail 상태 등 hardware event 를 관리한다.

5.2 RISC-V software 역할

- MMIO register 를 polling 하여 HuskyLens 상태와 좌표 데이터를 읽는다.
- xCenter 를 web game 화면에 맞는 범위로 scaling 한다.
- yCenter 의 이전 값과 현재 값을 비교하여 delta 를 계산한다.
- delta 와 위치 조건을 이용하여 RUN 또는 JUMP action 을 결정한다.
- Raspberry Pi 가 해석할 수 있는 4-byte game frame 을 생성하여 UART TX register 에 write 한다.

5.3 Raspberry Pi software 역할

- /dev/serial0 또는 지정된 serial port 에서 UART 데이터를 수신한다.
- 0xFF header 를 기준으로 4 바이트 frame 을 해석한다.
- x, delta, action 값을 Flask server state 에 반영한다.
- HTML Canvas 와 JavaScript 를 이용하여 게임 캐릭터, 장애물, 칼로리, 세트 진행률을 표시한다.
- controller/spectator 모드를 관리하여 여러 브라우저가 접속하더라도 한 명만 설정을 제어하도록 한다.

6. Agent AI 활용 내용

본 프로젝트에서는 AI Agent 를 단순한 코드 생성 도구가 아니라, 설계 보조, 디버깅, 문서화, 아이디어 확장 도구로 활용하였다. 프로젝트 진행 과정에서 ChatGPT 와 Codex 계열 도구를 사용하여 Verilog 설계, Raspberry Pi 서버 구축, UART 통신 구조, 보고서 작성 방향을 정리하였다.

초기 단계에서는 HuskyLens 로 사람의 위치를 인식하여 LED 로 LEFT, CENTER, RIGHT 를 표시하는 단순 시스템을 구상하였다. AI Agent 와의 대화를 통해 이 아이디어를 운동자 중앙 정렬 가이드, 포징 가이드, 피트니스 게임 입력 장치로 확장하였다. HuskyLens 가 관절 좌표를 제공하는 pose estimation sensor 가 아니라 block 중심 좌표를 제공하는 vision sensor 라는 한계를 고려하여, 정밀 자세 교정보다는 좌표 기반 움직임 입력 시스템이 현실적인 방향이라는 결론을 얻었다.

6.1 설계 및 프롬프트 작성

AI Agent 는 Codex 용 프롬프트 작성에 특히 유용하게 활용되었다. Verilog code 를 바로 생성하기보다, top module 이름, clock, baud rate, KEY input, GPIO 연결, UART frame, timeout, LED debug, Raspberry Pi 수신 코드까지 포함한 구체적인 요구사항을 프롬프트 형태로 정리하였다. 이를 통해 AI code generator 가 프로젝트 맥락을 잃지 않고 필요한 module 을 생성할 수 있도록 하였다.

6.2 디버깅 및 문서화

USB-Blaster driver 문제, top-level entity mismatch 문제, COM port 접근 문제, Prolific USB-RS232 driver 문제, PuTTY 와 Raspberry Pi SSH 접속 문제 등을 단계별로 정리하고 해결 방향을 도출하였다. 또한 Raspberry Pi 의 Flask server 구성, hotspot 환경에서의 IP 확인, wpa_supplicant 및 NetworkManager 설정, vim/nano 사용법 등 hardware 외적인 Linux 환경 문제 해결에도 AI Agent 를 활용하였다.

최종적으로 AI Agent 는 업로드한 Verilog, Python, memory initialization, SDC 파일을 기반으로 시스템 구조, RISC-V 활용 방식, MMIO memory map, sensor interface, hardware/software 역할 분담, 문제 해결 과정, 한계점 및 개선 방향을 보고서 항목에 맞게 정리하는 데 사용되었다. 단, AI 가 제안한

설계와 코드는 그대로 신뢰하지 않고, Quartus compile 결과, 실제 UART 출력, Raspberry Pi web 화면, sensor 동작 여부를 통해 검증하였다.

7. 구현 과정에서 발생한 문제와 해결 방법

7.1 USB-Blaster driver 문제

초기 FPGA programming 단계에서 Altera USB-Blaster 가 Windows 장치 관리자에서 Code 39 오류를 발생시켰다. 이는 오래된 USB-Blaster driver 가 Windows 보안 정책 또는 memory integrity 기능에 의해 차단되어 발생한 문제였다. 기존 driver 를 삭제한 뒤 Quartus 설치 폴더의 USB-Blaster driver 를 다시 지정하고, Windows 보안 설정을 조정하여 Programmer 에서 USB-Blaster 를 정상 인식시켰다.

7.2 Quartus top-level entity mismatch 문제

Quartus compile 과정에서 top-level design entity 가 undefined 라는 오류가 발생하였다. 원인은 Quartus 의 top-level entity 설정과 Verilog 파일 내부 module 이름이 일치하지 않았기 때문이다. 파일 이름이 아니라 module 이름이 중요하므로, 현재 구조에 맞게 TOP 또는 해당 wrapper module 이름을 설정하고 필요한 Verilog 파일을 모두 project 에 추가하였다.

7.3 DATAPATH latch 및 reset 조건 문제

DATAPATH 수정 과정에서 asynchronous reset 조건과 synchronous flush 조건이 섞이면서 latch warning 또는 elaboration error 가 발생할 수 있었다. RESET 은 asynchronous reset 조건으로만 처리하고, ex_flush, hazard_stall 등은 clock edge 이후의 synchronous 조건으로 분리하여 문제를 해결하였다.

7.4 UART TX/RX 연결 혼동

UART 연결에서 TX 와 RX 를 같은 이름끼리 연결하려는 혼동이 있었다. UART 는 송신과 수신을 교차 연결해야 하므로, HuskyLens TX 는 FPGA RX 역할의 GPIO_0[0]에 연결하고, FPGA TX 인 GPIO_0[1]은 HuskyLens RX 에 연결해야 한다. Raspberry Pi 와 연결할 때도 FPGA TX 는 Raspberry Pi RXD 인 GPIO15, physical Pin 10 에 연결해야 한다.

7.5 RS-232 와 TTL UART 혼동

DE2 보드의 DB9 RS-232 포트와 HuskyLens/Raspberry Pi GPIO 의 TTL UART 는 전압 레벨이 다르다. RS-232 를 Raspberry Pi GPIO 나 HuskyLens 에 직접 연결하면 통신이 되지 않거나 장치가 손상될 수 있다. 따라서 HuskyLens 와 Raspberry Pi 는 FPGA GPIO 기반 TTL UART 로 연결하거나, DB9 RS-232 를 사용할 경우 변환기를 거쳐야 한다.

7.6 COM port 및 USB-to-RS232 문제

PC 에서 MobaXterm 으로 UART 출력을 확인하려 했지만 COM port 가 존재하지 않거나 접근 거부 오류가 발생하였다. 일부 USB-to-RS232 변환기는 Prolific driver 호환성 문제를 보였다. 대안으로 FTDI, CH340, CP2102 계열 변환기를 사용하거나 Raspberry Pi 를 serial receiver 로 사용하도록 구성하였다.

7.7 Raspberry Pi SSH 및 네트워크 문제

Wi-Fi 비밀번호와 Linux 계정 비밀번호를 혼동하는 문제가 있었다. wpa_supplicant.conf 의 psk 는 Wi-Fi 접속 비밀번호이고, PuTTY 에서 요구하는 비밀번호는 Raspberry Pi 사용자 계정 비밀번호이다. 또한 hotspot 환경에서는 `arp -a`, `raspberrypi.local`, `hostname -I`, `nmcli` 등을 활용하여 IP 를 찾았다.

7.8 Flask server 실행 및 터미널 사용 문제

`python3 server.py` 를 실행하면 Flask server 가 foreground 에서 실행되므로 같은 PuTTY 창에서 추가 명령을 입력할 수 없다. 추가 작업이 필요하면 `Ctrl+C` 로 서버를 종료하거나 PuTTY 창을 하나 더 열어야 한다. vim 이 설치되어 있지 않은 경우에는 `nano` 를 사용하거나 `sudo apt install vim` 으로 설치하였다.

7.9 HuskyLens 인식 안정성 문제

Object Tracking 모드는 실환경에서 구도와 조명 변화에 민감했다. 이를 해결하기 위해 Face Detection 모드를 우선 사용하고, 카메라 방향은 가로, 권장 거리는 0.5m~1.5m 로 설정하였다. Block 이 일시적으로 미감지될 경우 마지막 유효 좌표를 latch 하고 이전 action 을 유지하도록 보완하였다.

7.10 Jump frame 누락 문제

초기에는 FPGA 가 짧은 순간 `action=1` frame 을 보냈을 때 웹 브라우저 polling 이 이를 놓치는 문제가 있었다. Python 수신부에 `jump_events` 누적 카운터를 추가하여 action frame 이 짧게 지나가도 웹 UI 가 jump event 증가를 감지할 수 있도록 개선하였다.

8. Demo 결과

Demo 는 크게 세 단계로 진행하였다. 첫 번째 단계에서는 FPGA 와 HuskyLens 간 UART 통신을 확인하였다. KEY[1]을 누르면 FPGA 가 HuskyLens 에 knock command 를 전송하고, HuskyLens 가 OK response 를 반환하는지 확인하였다. 이후 request command 를 보내고 xCenter, yCenter block data 가 정상적으로 parser 에 들어오는지 확인하였다.

두 번째 단계에서는 RISC-V processor 가 MMIO 를 통해 HuskyLens 좌표를 읽고, UART TX register 를 통해 4 바이트 frame 을 전송하는지 확인하였다. RISC-V software 는 status, xCenter, yCenter, TX busy register 를 polling 하고, TX data register 에 frame byte 를 순서대로 write 하였다.

세 번째 단계에서는 Raspberry Pi 에서 Flask web game 을 실행하고, 브라우저를 통해 game UI 를 확인하였다. 사용자가 HuskyLens 앞에서 좌우로 움직이면 캐릭터의 x 위치가 변하고, y 방향으로 빠르게 이동하면 jump action 이 발생하였다. 웹 UI 에는 현재 UART 연결 상태, x 값, delta 값, action 값, frame count, jump event count, 칼로리, 세트 진행률, 현재 동작 상태가 표시되었다.

입력 동작	FPGA/RISC-V 처리	Web UI 결과
좌우 이동	xCenter 변화 → X_scaled 계산 → frame 전송	캐릭터 좌우 이동
빠른 상하 움직임	yCenter delta 계산 → action=JUMP 또는 jump_events 증가	캐릭터 점프
Block 미감지	No_Person 또는 이전 action 유지	WAITING 또는 이전 상태 유지
UART 실패 또는 센서 미연결	error 상태 또는 frame 미수신	WAITING 표시, keyboard demo mode 사용 가능

또한 keyboard demo mode 를 제공하여 FPGA 또는 HuskyLens 가 연결되지 않은 상황에서도 웹 게임 UI 를 테스트할 수 있도록 하였다. 사용자는 방향키와 Space/Up 키를 이용해 캐릭터를 조작할 수 있으며, 이를 통해 web UI, 장애물 생성, 점수 계산, 세트 진행 로직을 독립적으로 검증할 수 있었다. Quartus 합성 결과는 0 errors 로 완료되었으며, 50 MHz 기준 timing slack 도 충분히 확보되었다.

9. 한계점 및 개선 방향

9.1 한계점

- HuskyLens 는 정밀 pose estimation sensor 가 아니므로 무릎, 허리, 팔꿈치 각도와 같은 관절 기반 자세 분석은 어렵다.
- 거리, 조명, 배경, 얼굴 방향에 따라 block 인식이 불안정할 수 있다.
- 현재 UART frame 은 단순 4 바이트 구조이므로 checksum, sequence number, timestamp 가 포함되어 있지 않다.
- RISC-V software 가 assembly 또는 machine code 중심으로 구성되어 있어 C 기반 개발에 비해 수정이 번거롭다.
- DMEM 활용이 제한적이며, 게임 상태 저장이나 sensor history 저장 기능은 충분히 확장되지 않았다.
- Raspberry Pi 와 FPGA 간 통신이 현재는 FPGA→Raspberry Pi 단방향 중심이다.

9.2 개선 방향

- 색상 마커 기반 multi-object tracking 을 추가하여 손, 머리, 몸 중심을 분리 추적한다.
- RISC-V GCC toolchain 과 linker script 를 구성하여 C 기반 application logic 을 작성한다.
- DMEM 을 활용하여 장애물 정보, 점수, frame history, calibration table 을 저장한다.
- UART frame 에 checksum 과 sequence number 를 추가하여 데이터 신뢰성을 높인다.

- Raspberry Pi 에서 FPGA 로 control command 를 보내는 bidirectional UART 구조를 구현한다.
- 웹 UI 에 실시간 그래프를 추가하여 xCenter, yCenter, delta 변화를 시각화한다.
- 운동별 mode 를 분리하여 점프 모드, 스쿼트 모드, 포징 모드 등으로 확장한다.

10. 팀원별 역할 분담

역할	담당 내용
차상훈: FPGA/RISC-V Hardware 및 Vision Sensor Interface, game system & UI design, demo 영상 제작	TOP.v 에서 RISC-V, IMEM, DMEM, MMIO, HuskyLens core 를 연결하고 Quartus project 설정, top-level entity 지정, pin assignment, SDC timing constraint 적용을 담당하였다. 또한 DATAPATH.v 의 5-stage pipeline, forwarding, load-use stall, branch flush 로직을 검토하고, BRANCH_PRED.v 의 2-bit BHT dynamic predictor 적용을 담당하였다. HuskyLens 와 FPGA 간 UART 통신을 위해 huskylens_core.v 의 knock command, OK response, request command, response frame 수신, timeout 처리, block data 추출 구조를 구현 및 검증하였다. 게임 소프트웨어 및 시스템 구현 및 검증, 게임 UI 업그레이드를 추가적으로 진행하였다. Demo 시나리오 촬영을 추가 진행하였다.
팀원 [REDACTED]: MMIO/RISC-V Software, Raspberry Pi Server 및 Demo 담당	MMIO.v 에서 HuskyLens status, xCenter, yCenter, UART busy, UART data register 를 구성하고, RISC-V software 가 해당 register 를 이용해 sensor data 를 읽고 frame 을 생성하도록 설계하였다. 또한 Raspberry Pi 5 와 FPGA 간 UART 통신 구조를 설정하고, rpi_uart_receive.py 를 이용해 UART frame 수신, pyserial parsing, Flask server, HTML Canvas 기반 game UI 를 구현하였다. PuTTY SSH 접속, hotspot/network setup, Flask server 실행, demo 시나리오 구성 및 presentation, 보고서 정리를 담당하였다.

10.1 통합 역할

팀원	통합 역할
차상훈	FPGA/RISC-V hardware 담당: TOP.v 에서 RISC-V, IMEM, DMEM, MMIO, HuskyLens core 를 연결하고 Quartus project 설정, top-level entity 지정, pin assignment, SDC timing constraint 적용을 담당하였다. 또한 DATAPATH.v 의 5-stage pipeline, forwarding, load-use stall, branch flush 로직과 BRANCH_PRED.v 의 2-bit BHT dynamic predictor 적용을 검토하였다.
[REDACTED]	HuskyLens/MMIO/UART 및 Raspberry Pi 연동 담당: huskylens_core.v 의 parser, knock command, response frame 수신 구조를 검토하고, MMIO.v 를 통해 HuskyLens status, xCenter, yCenter, UART busy, UART data register 가 RISC-V software 와 연결되도록 구성하였다. 또한 Raspberry Pi 5 와의 UART frame format, pin connection, Flask server

및 demo UI 연동을 담당하였다.

본 프로젝트는 각 모듈이 독립적으로 존재하는 것이 아니라 sensor input 부터 web output 까지 연결되는 end-to-end system 이다. 따라서 팀원 간 interface 정의가 매우 중요했다. 특히 UART frame format, MMIO address map, GPIO pin assignment, action encoding, Raspberry Pi 수신 format 을 명확히 맞추는 것이 통합 성공의 핵심이었다.